JAVASCRIPT ESSENTIALS

Core Concept Cheat Sheet & Reference Notes

1. VARIABLES & SCOPE

JavaScript has three ways to declare variables, each handling scoping differently:

- `var` : Function-scoped, hoisted to the top of its scope, initialized as `undefined` . Can be redeclared.
- `let` : Block-scoped `{ }` , hoisted to the "Temporal Dead Zone" (unaccessible until initialized). Cannot be redeclared.
- `const` : Block-scoped like `let` , but must be assigned a value immediately and cannot be reassigned.

```
// Scope Breakdown
if (true) {
    var globalLike = "Accessible outside block";
    let blockOnly = "Hidden outside block";
}
```

2. DATA TYPES

JS is dynamically typed and divided into two categories:

Primitives (Passed by Value)

- `String` : 'Hello'
- `Number` : Integer or Float (e.g., 42, 3.14)
- `Boolean` : true / false
- `Null` : Intentional absence of value
- `Undefined` : Unassigned variable
- `Symbol` / `BigInt`

Objects (Passed by Reference)

- `Object` : `{ key: "value" }`
- `Array` : `[1, 2, 3]`
- `Function` : Callable object chunks

3. FUNCTIONS & ARROW SYNTAX

Functions can be declared or defined as expressions. Arrow functions offer a shorter syntax and lexical `this` binding.

```
// Declaration
function greet(name) { return `Hello ${name}`; }

// Arrow Function (implicit return for single expression)
const greetArrow = name => `Hello ${name}`;
```

> **Important:** Arrow functions do not have their own `this` context; they inherit it from the surrounding code block.

4. DOM MANIPULATION

The Document Object Model (DOM) connects web pages to scripts.

```
// Selecting elements
const element = document.querySelector('.my-class');

// Updating content & styles
element.textContent = "New Text Content";
element.style.color = "blue";

// Adding event listeners
element.addEventListener('click', (e) => {
    console.log('Element clicked!', e.target);
});
```

5. ARRAY METHODS (FUNCTIONAL)

Modern JS relies heavily on clean iteration methods instead of traditional loops:

- `.map()` : Creates a new array by transforming every element.
- `.filter()` : Creates a new array with elements that pass a test condition.
- `.reduce()` : Reduces the array to a single value (e.g., summing numbers).
- `.forEach()` : Executes a function once for each array element (no return value).

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2); // [2, 4, 6, 8]
const evens = nums.filter(n => n % 2 === 0); // [2, 4]
```

6. ASYNCHRONOUS JAVASCRIPT

Handles operations that take time (API requests, timers) without freezing the UI thread.

Promises

An object representing the eventual completion or failure of an async operation.

```
fetch('https://api.example.com/data')
    .then(response => response.json())
    .catch(error => console.error('Error:', error));
```

Async / Await

Syntactic sugar built on top of Promises, making async code read like synchronous code.

```
async function fetchData() {
    try {
        const response = await fetch('https://api.example.com/data');
        const data = await response.json();
        return data;
    } catch (err) {
        console.error("Fetch failed", err);
    }
}
```